

PX4 SITL + Gazebo Drone Simulation — Detailed Session Report

June 17, 2026

1. Session Overview

Today's session covered end-to-end troubleshooting and progress on a PX4 + Gazebo Harmonic SITL drone simulation running inside WSL2 on Windows, with QGroundControl (QGC) as the ground control station. Work spanned networking diagnostics, environment/toolchain repair, flight testing, Python-based offboard control via MAVSDK, early multi-drone setup, and planning for a custom swarm intelligence algorithm (SMA + Stigmergy) to be implemented and tested in this environment.

2. PX4 ↔ QGroundControl Connectivity Fix

2.1 Symptom

QGroundControl displayed "Disconnected" despite PX4 and Gazebo running correctly in WSL. PX4 repeatedly logged:

```
WARN [health_and_arming_checks] Preflight Fail: No connection to the GCS
```

This blocked arming, since PX4's safety checks require an active GCS heartbeat.

2.2 Diagnostic Path

- Confirmed a Windows Firewall inbound rule ("PX4 GCS Fix") existed for UDP port 14550, allowing traffic on all profiles — ruled out as the blocker.
- Confirmed via netstat that QGroundControl was actively listening on UDP 14550 on the Windows side.
- Ran mavlink status repeatedly in the PX4 shell, which showed healthy tx (transmit) rates on all four MAVLink instances, but rx (receive) consistently at 0.0 B/s — confirming PX4 was sending telemetry out, but never receiving a heartbeat back from QGC.
- Attempted netsh interface portproxy for UDP forwarding — found this Windows utility only supports TCP, not UDP, ruling it out.
- Checked WSL networking mode via .wslconfig — found no config file existed, confirming WSL was running in default NAT mode rather than mirrored mode.
- Identified that in NAT mode, inbound UDP initiated from Windows back into the WSL virtual subnet is not automatically routed, explaining the one-way tx/rx symptom.

2.3 Resolution

Created a .wslconfig file enabling mirrored networking mode, which shares the host's network stack directly with WSL rather than placing it behind a NAT boundary:

```
[wsl2]
```

```
networkingMode=mirrored
```

Restarted WSL fully (wsl --shutdown) to apply the change. Also identified that the Wi-Fi network profile was set to "Public" in Windows, which applies stricter implicit firewall behavior; changed it to "Private" via:

```
Set-NetConnectionProfile -InterfaceAlias "Wi-Fi" -NetworkCategory Private
```

After both changes and a clean PX4/Gazebo relaunch with PX4_SIM_HOST_ADDR set to the loopback/host address, QGroundControl successfully connected, GCS preflight warnings cleared, and arming became possible.

3. Gazebo Binary Conflict ("gz sim not found")

3.1 Symptom

PX4 build failed during simulation launch with:

```
ERROR [init] Gazebo gz sim not found. Please install gz-harmonic
```

despite gz being present and resolvable on the system PATH.

3.2 Root Cause

Investigation via pixi global list revealed two separate, conflicting pixi-managed environments: an old Gazebo Classic 11.15.1 install (which exposed a gz command) and a Gazebo Harmonic 1.0.0 install (which exposed nothing). The old Classic gz binary was shadowing the correct Harmonic one on PATH, and Classic's gz command does not implement the sim subcommand PX4 depends on.

A secondary discovery during this process: the OS was identified as Ubuntu 26.04 ("Resolute Raccoon"), a release new enough that lsb_release returned a non-standard codename and the official OSRF apt repository for Gazebo Harmonic was not yet available for it, causing earlier apt-based install attempts to fail with unresolvable Python version dependency conflicts (gz-harmonic depended on python3 < 3.13, incompatible with the system's Python 3.13+).

3.3 Resolution

```
pixi global remove gazebo
```

```
pixi global expose add gz=gz --environment gz-harmonic
```

This removed the conflicting Classic exposure and explicitly exposed the correct Harmonic gz binary. Verified successfully:

```
$ gz sim --version
```

```
Gazebo Sim, version 8.10.0
```

4. Model File Corruption (x500/model.sdf)

4.1 Symptom

After saving the simulation world from inside the Gazebo GUI, the drone stopped spawning entirely. PX4 logged:

[Err] [UserCommands.cc:928] Error Code 8: Failed to find top level <model> / <actor> / <light> for <include>

and the Gazebo Entity Tree showed only world elements (park, coast_waves, sunUTC) with no x500_0 drone entity present.

4.2 Root Cause

Using Gazebo's GUI "Save" functionality had overwritten and corrupted the tracked source file Tools/simulation/gz/models/x500/model.sdf, breaking the <include> resolution that PX4's launch process depends on. Confirmed via:

```
$ git status --porcelain | grep -i x500
```

```
M models/x500/model.sdf
```

4.3 Resolution

Restored the original tracked file directly from git history:

```
cd ~/robotics_sim/PX4-Autopilot/Tools/simulation/gz
```

```
git checkout -- models/x500/model.sdf
```

Established a standing rule going forward: never use Gazebo's GUI "Save" on PX4's original tracked model/world files. Any future saves should use "Save World As..." targeting a separate custom directory outside the tracked PX4 source tree.

5. Flight Testing

With connectivity and the model file restored, a full arm → takeoff → hover → land → disarm cycle was successfully completed in the baylands world using PX4 shell commands:

```
commander arm
```

```
commander takeoff
```

```
commander land
```

```
commander disarm
```

Notable behaviors confirmed during testing: commander arm produces no output if the vehicle is already armed (not an error — simply no state change); commander disarm is correctly rejected mid-air with "not landed" until the vehicle's landing detector confirms touchdown, after which it auto-disarms. Vehicle state was cross-checked using commander status and listener vehicle_local_position.

One incident occurred where the drone drifted into a tree collision mesh in the baylands world and became physically stuck; recovery options covered included manually editing the model's Pose values in the Gazebo Entity Tree panel, or performing a full clean relaunch to respawn the vehicle at its default home position.

Attempted to change default takeoff altitude via the parameter MPC_TAKEOFF_ALT, which does not exist in the installed PX4 version ("Parameter not found"). Redirected to search via param show for

the correct current parameter name, most likely MIS_TAKEOFF_ALT in current PX4 releases, pending final confirmation.

6. MAVSDK (Python) Offboard Control Setup

Installed MAVSDK-Python inside the WSL environment to enable programmatic drone control outside of manual shell commands:

```
pip install mavsdk --break-system-packages
```

Confirmed successful import under Python 3.14:

```
$ python3 -c "from mavsdk import System; print('OK')"
```

OK

Built and ran a test offboard control script (test_mavsdk.py) that connects to PX4, arms the vehicle, starts offboard mode, and commands a position setpoint in NED (North-East-Down) coordinates. Two issues were resolved along the way: a deprecation warning requiring the explicit udpin:// (listen) or udpout:// (send) address scheme instead of the legacy udp:// scheme, and an "Address in use" bind error caused by a port conflict, resolved by binding explicitly to udpin://0.0.0.0:14540. The script was also adjusted to test higher-altitude flight by increasing the magnitude of the negative Z value in the NED setpoint (e.g., -15 instead of -5, since Down is negative in NED).

7. Multi-Drone Setup (In Progress)

Began configuring a second PX4 instance to run alongside the first within the same Gazebo world, using the instance-flag approach:

```
PX4_SIM_MODEL=x500 PX4_GZ_WORLD=baylands PX4_SIM_HOST_ADDR=127.0.0.1 \
```

```
PX4_INSTANCE=1 ./build/px4_sitl_default/bin/px4 -i 1
```

Initial attempt failed with "Unknown model x500", since instance-based startup reads PX4's internal airframe registry rather than the make target naming scheme. Diagnostic next step identified: list the airframes directory under the instance's rootfs to find the exact registered model name to use in PX4_SIM_MODEL. This remains an open item to finish validating before scaling to full simultaneous two-drone control.

Planned approach for true simultaneous control once both instances spawn correctly: a Python script using MAVSDK's `asyncio.gather()` to run independent control coroutines for each drone in parallel, each connected to its own instance's MAVLink port.

8. Environment / World Exploration

Diagnosed a separate issue where Gazebo appeared to hang indefinitely on "Waiting for Gazebo world...", traced to an "Unable to find or download file" message suggesting a missing or unreachable model asset; resolved by relaunching with GZ_SIM_RESOURCE_PATH set explicitly to the local model/world directories.

Reviewed available PX4-supported Gazebo Harmonic worlds beyond baylands, including a minimal lawn world, a rover-optimized flat world, and an empty windy world with wind physics enabled but no scenery. Successfully launched and tested the windy world, confirmed via its Entity Tree (ground_plane, sunUTC, x500_0 only) and a notably higher real-time factor (92% vs. baylands' typical ~50%) due to its lack of complex meshes.

Discussed the process for building a fully custom Gazebo world via hand-written SDF, including placing obstacle models at exact coordinates to match the bounds and obstacle layout required by the planned swarm algorithm, and launching it either through PX4's world-name convention or in standalone mode (gz sim alongside a separately launched PX4_GZ_STANDALONE=1 instance).

9. SMA + Stigmergy Swarm Algorithm — Implementation Planning

Reviewed a complete technical specification for a hybrid swarm intelligence algorithm combining the Slime Mould Algorithm (SMA) with ant-colony-style stigmergy (a shared, decaying, diffusing pheromone field), designed specifically for jamming-resilient multi-drone coordination. Its key novelty is a jamming-aware position update: agents that lose access to the swarm's global best reference fall back to local self-reference, while the pheromone field — modeled as a local physical signal rather than a communication channel — remains sensible even in jammed zones, providing decentralized fallback coordination.

Laid out a seven-step implementation plan:

1. Validate the MAVSDK control interface independently.
2. Build and test the SMA + stigmergy logic as a standalone 2D Python simulation before touching any drone hardware/sim interface.
3. Map the algorithm's abstract coordinate space to real NED coordinates matching the Gazebo world's bounds.
4. Validate the full per-iteration control loop on a single drone.
5. Scale to multiple drones once multi-instance launching is confirmed working.
6. Simulate communication jamming as a software-defined zone check within the Python logic rather than any real RF disruption.
7. Implement the pheromone field as a shared in-memory grid (e.g., a numpy array) accessible to all drone control loops within the same process.

10. ROS 2 — Role and Relevance

Discussed where ROS 2 would fit relative to the current MAVSDK-based approach. Key points: PX4 already runs a uXRCE-DDS bridge by default (visible in every session's startup log), which is the mechanism ROS 2 would use to communicate with PX4 as an alternative to MAVSDK. ROS 2 becomes most valuable for this specific project if pursuing a more architecturally faithful decentralized swarm simulation, where each drone runs as a fully independent node communicating only through the shared pheromone field — better matching the algorithm's real-world jamming-resilience claims

than a single centralized Python script. Recommendation given: defer ROS 2 adoption until the core SMA + Stigmergy algorithm logic is validated using the simpler MAVSDK approach already in place.

11. Current Status Summary

Item	Outcome
GCS Connectivity	Resolved — mirrored WSL networking mode + Private network profile
Gazebo Toolchain	Resolved — correct Harmonic gz binary exposed via pixi
Drone Model File	Resolved — restored from git, corruption cause identified and avoided going forward
Single-Drone Flight	Working — arm/takeoff/land/disarm cycle confirmed functional
Takeoff Altitude Parameter	Open — correct parameter name (likely MIS_TAKEOFF_ALT) needs confirmation
MAVSDK Offboard Control	Working — single-drone position setpoint script functional
Multi-Drone Setup	In progress — second instance airframe naming issue needs resolution
Custom World Building	Planned — approach defined, not yet built
SMA + Stigmergy Algorithm	Planned — full specification reviewed, implementation not yet started

12. Recommended Next Steps

1. Confirm the correct takeoff-altitude parameter name via param show and set a working test altitude.
2. Resolve the second PX4 instance's airframe naming issue to unblock true multi-drone testing.
3. Build the standalone 2D Python simulation of the SMA + Stigmergy algorithm (no drone dependency) to validate its mathematical behavior before integrating with MAVSDK.
4. Once validated, connect the algorithm's output (target positions) to the existing MAVSDK control script for single-drone, then multi-drone, closed-loop testing.
5. Construct a custom minimal Gazebo world with hand-placed obstacles matching the algorithm's intended test scenario, to replace baylands for repeatable, performance-friendly experiments.